Add 算子测试报告

一、算子理解

本测试覆盖的算子为 Add 系列（包含 aclnnAdd、aclnnAddV3 及其 inplace 版本 aclnnInplaceAdd、aclnnInplaceAddV3），对应数学运算：

out=self+α×other

核心版本说明

- **张量版本 (aclnnAdd)**：self 和 other 均为张量，支持广播，输出张量形状由广播规则决定。
- **张量-标量版本 (aclnnAdds)**：self 为张量，other 为标量；输出形状与 self 相同。
- **标量-张量版本 (aclnnAddV3)**：self 为标量，other 为张量；输出形状与 other 相同。
- **原地版本**：self 张量同时作为输出，其内容被修改。

输入输出规格

维度	说明
输入	self（张量 / 标量）、other（张量 / 标量）、alpha（标量）
输出	与广播结果形状一致的张量
支持 dtype	浮点：FLOAT、FLOAT16、BFLOAT16、DOUBLE；整数：INT8、INT32（部分支持 UINT8）；布尔：BOOL
形状约束	秩最大为 8（超 8 返回 ACLNN_STATUS_PARAM_INVALID）；广播需兼容

定义域与边界行为

类型	行为规则
浮点运算	遵循 IEEE 754；NaN 传播、±inf 相抵为 NaN、溢出生成 ±inf/denormal/ 零
整数运算	溢出按 C/C++ 固定宽度环绕（wrap-around）
布尔类型	0.0 视为 false，非 0 为 true；加法等价于逻辑或（硬件可能按整数饱和处理）
标量 alpha	允许 0.0（仅输出 self）、负数（实现减法）
空张量	维度为 0 时元素总数为 0，验证不崩溃且返回空输出

特殊性质

- 可广播、alpha=1 时可交换；
- 支持 inplace 计算，降低显存占用；
- 多态接口：通过 GetWorkspaceSize 查工作空间，aclnnAdd 等在流上执行运算。

二、测试策略与用例设计

测试方法思路

- **参照实现（Oracle）**：CPU 端纯 C++ 实现等价运算（类型转换、广播、alpha 乘加）；
- **精度验证**：设备输出拷回主机，逐元素与 Oracle 比较（按 dtype 设公差）；

- **测试框架**：单文件自包含，无外部依赖，通过 LOG_PRINT 输出、CHECK_RET 处理错误，统计通过率。

用例分类与分布

分类	覆盖场景
基本功能	FLOAT32 普通加法、广播、alpha=0 / 负 alpha、大尺寸（1024×128）
不同精度	FLOAT16、BFLOAT16 加法，验证精度容差
混合精度	FLOAT16+FLOAT32、FLOAT32+BFLOAT16，验证类型提升与结果正确性
整数溢出	INT32（±2^31）、INT8（±128）极限值加法，验证环绕行为
布尔运算	BOOL 张量加法、BOOL 标量输出到 FLOAT16 的特殊路径
高维与广播	秩 8 张量、广播维度为 1 的极端情况
特殊值	NaN、±Inf、浮点极值、denorm、大整数，验证异常传播与精度边界
Inplace 操作	覆盖 InplaceAdd/InplaceAdds，验证 self 写回正确性
V3 接口	标量 + 张量（AddV3/InplaceAddV3），覆盖 INT8/BFLOAT16/FLOAT16 等路径
无效输入	空指针、不支持 dtype、形状不兼容、秩 > 8 等，验证错误码正确性

阈值设定依据

类型	公差规则
浮点类型	FLOAT (1e-6)、FLOAT16 (1e-4)、BFLOAT16 (1e-2)（相对 / 绝对误差组合）
整数 / 布尔	严格相等（公差 0），因运算位模式完全确定

三、覆盖率分析

测量方法

使用 gcov -b 统计行覆盖率 + 分支覆盖率，核心命令：

```
gcov -b $(find build -name "aclnn_add.cpp.gcda" | head -1) 2>&1 | grep -A2 "File.*aclnn_add.cpp" | head -3
gcov -b $(find build -name "aclnn_add_v3.cpp.gcda" | head -1) 2>&1 | grep -A2 "File.*aclnn_add_v3.cpp" | head -3
gcov -b $(find build -name "add.cpp.gcda" | grep "add/op_api" | head -1) 2>&1 | grep -A2 "File.*op_api/add.cpp" | head -3
gcov -b $(find build -name "add_tiling_arch35.cpp.gcda" | head -1) 2>&1 | grep -A2 "File.*add_tiling" | head -3
```

覆盖率结果

文件	行覆盖率	分支覆盖率
op_api/aclnn_add.cpp	66.34% (303 行)	41.85% (1546 分支)
op_api/aclnn_add_v3.cpp	88.31% (77 行)	48.36% (426 分支)
op_api/add.cpp	55.93% (59 行)	23.48% (264 分支)
op_host/arch35/add_tiling_arch35.cpp	89.25% (93 行)	57.29% (192 分支)
综合加权	**≈70.4%**	**≈44.2%**

未覆盖部分分析

未覆盖类型	归因
错误处理路径	参数校验分支多（dtype / 形状 / 秩等），仅覆盖部分非法参数组合
特化架构路径	arch35 tiling 策略分支仅在特定 shape/dtype 下触发，用例未全覆盖
V3 接口多 dispatch	dtype 组合分发路径未全覆盖（如 FLOAT16+INT32 不支持组合仅返回错误）
空张量 / 极端形状	add.cpp 仅覆盖 INT32 空张量，未覆盖全 dtype 空张量、复杂广播形状

四、精度分析

误差度量与阈值

- **度量公式：** |actual - expected| ≤ atol + rtol × |expected|（atol/rtol=0 时要求完全相等）；
- **阈值表：**

dtype	atol	rtol
FLOAT	1e-6	1e-6
FLOAT16	1e-4	1e-4
BFLOAT16	1e-2	1e-2
DOUBLE	1e-12	1e-12
整数 / BOOL	0	0

Oracle 实现依据

- 浮点：模拟目标格式存储转换（half/bfloat）；
- 整数：模截断实现溢出环绕；
- 布尔：非零→1.0、零→0.0，加法后截断；

- 特殊值：精确匹配 NaN/Inf 类别（std::isnan/std::isinf）。

典型精度场景验证结果

场景	验证结论
常规浮点	误差远小于阈值，IEEE 754 操作无额外误差
半精度 / 脑浮点	量化误差在容差内，无可见偏差
浮点溢 / 下溢	Inf/denorm/ 零生成符合标准，与 Oracle 一致
整数溢出	INT32 (2147483647+1→-2147483648)、INT8 (127+1→-128) 精确匹配
布尔饱和	1+1→1、0+0→0，误差为零
NaN/Inf 传播	特殊值传播规则完全符合预期
非精确小数	FLOAT32 无误差，FLOAT16 量化偏差在 1e-4 内

精度不达标情况

本次所有用例均通过，无精度不达标情况，阈值符合 CANN 算子规范。

五、反思与改进

测试盲区与局限性

1. 随机性不足：输入为固定公式生成，缺少大规模随机形状 / 数值模糊测试；
2. 广播覆盖不全：未覆盖复杂维度排列的广播场景（如 [2,1,3,1]+[1,4,1,5]）；
3. 张量属性缺失：未测试非连续 / 负 stride 输入；
4. 并发 / 内存：单 stream 测试，未测多 stream 并发、内存受限场景；
5. 硬件单一：仅在 arch35 平台验证，未覆盖其他架构。

扩展优化方向

1. 引入随机测试生成器，自动对比 PyTorch/TensorFlow 参考结果；
2. 系统性遍历 dtype / 广播形状，覆盖所有参数校验分支；
3. 补充非连续 stride、全 dtype 空张量、超大张量鲁棒性测试；
4. 多卡环境测试通算融合精度；
5. 用 aclnnDebug/Profiling 工具做性能回归与内存检查。

方法论经验教训

1. Oracle 需严格模拟硬件行为（整数环绕、浮点量化、布尔特殊逻辑）；
2. 类型转换需位精确（用 memcpy 实现 float↔half/bfloat 转换）；
3. 公差区分类型：整数严格相等，半精度 / BF16 按需放大容差。

对 CANN 测试工具链的建议

1. 提供参数校验用例自动生成工具，提升分支覆盖率；
2. 细化错误码，增加调试日志，降低排查难度；
3. 提供 Host 端张量计算库，减少 Oracle 编写错误；
4. 集成 CI 覆盖率可视化，标注未覆盖分支条件。